CIS 531 **Name:** _____
Fall 2025 **PRACTICE Midterm 1**
Default Time Limit: 80 minutes

- You may not use anything except for a single **one-sided** *handwritten (by you)* note sheet of US letter size (8.5 x 11 inches) paper. You may not collaborate with anyone, Google for answers, use ChatGPT, etc...
- We will not penalize small syntactic issues (judged by instructor); you should document all of your relevant thoughts for potential partial credit—we will grade as consistently as possible, and will attempt to award partial credit when possible.

Problem	Points	Score
1	10	
2	10	
3	10	
4	10	
Total:	40	

1. **Syntax Trees and Interpreters in Racket**

In this question, you will build an interpreter for the following expression language:

```
(define (expr? e)
  (match e
    [(? symbol? x)  #t] ;; variables 'x
    [(? integer? i) #t] ;; integer literals 42
    [(? string? s)  #t] ;; string literals "42"
    [`(+ ,(? expr?) ,(? expr?))   #t] ;; adding two expressions
    ;; let bindings require *two* variables / expressions
    [`(let ([,(? symbol?) ,(? expr?)]
            [,(? symbol?) ,(? expr?)])
        ,(? expr?)) #t]))
```

You will do this by completing the following definition of the function interp:

```
;; expr satisfies expr?.  env is a hash, use (hash-ref env x) to lookup the
;; value associated with the key x, and use (hash-set env x v) to get a new hash
;; back, the same as env except x |-> v
(define (interp expr env)
  (match expr
    [(? symbol? x)  (hash-ref env 'x)]
    [(? integer? i) i]
    [(? string? s)  s]
    [`(+ ,e0, e1)
     'todo-PLUS]
    [`(let ([,x0 ,e0] [,x1 ,e1]) ,e)
     'todo-LET]))
```

(a) (5 points) Complete 'todo-PLUS, which should work in the following way: first, it should evaluate each of e0/e1. If their interpretations are both strings (check using string?) (v0/v1), the result should be the concatenation of v0 and v1 using string-append. If they are both integers (check with integer?) the results should simply be added. If they any other combination of types, you should raise an error with (error ...)

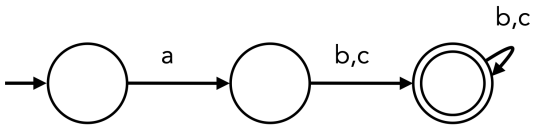
```
(match `(,(interp e0 env) ,(interp e1 env))
       [`(,(? integer? i0) ,(? integer? i1))
        (+ i0 i1)]
       [`(,(? string? s0) ,(? string? s1))
        (string-append s0 s1)]
       [_ (error "bad +")])
```

(b) (5 points) Complete todo-LET, which should work in the following way: first evaluate e0 and e1 (the binding x0 should not be visible during the evaluation of e1), and then build a new updated environment (using hash-ref, possibly more than once—it is **not** mutable!) in which to evaluate e.

```
(define v0 (interp e0 env))
(define v1 (interp e1 env))
(interp e (hash-set (hash-set env x0 v0) x1 v1))
```

2. **Regular Expressions**

(a) (3 points) Write a regular expression that corresponds to the following DFA:



`a(b|c)(b|c)*`

(b) (3 points) Consider the regular expression `(b(cc)*d)*`. Write three strings (all whose length is over five characters) which match the regular expression

`bccccccd`
`bccdbccd`
`bdbdbd`

(c) (4 points) Consider the following pseudocode, which uses the functions `peek()` and `consume(t)` (for some token t). The function ultimately returns `True` (indicating the pattern matched) or `False` (indicating the pattern didn't match).

```
def matcher():
  if (peek == '-'):
    consume('-')
    return matcher()
  if (peek == '0'):
    consume('0')
    return True
  else:
    return False
```

What pattern is matched by `matcher`?

Answer: -*0

3. **Grammars and Productions**

In this question we will consider the following grammar:

$$S \rightarrow S\ o\ S \mid S\ a\ S \mid e \tag{1}$$

The grammar is ambiguous, a fact which you will now prove by writing two distinct leftmost derivations of the string `eaeoeoe`

(a) (5 points) Write one leftmost derivation of `eaeoeoe`

```
S
=> S o S
=> S a S o S
=> e a S o S
=> e a e o S
=> e a e o S o S
=> e a e o e o S
=> e a e o e o e
```

(b) (5 points) Write another leftmost derivation of `eaeoeoe`

```
S
=> S a S
=> e a S
=> e a S o S
=> e a S o S o S
=> e a e o S o S
=> e a e o e o S
=> e a e o e o e
```

4. **Recursive Descent Parsers**

In this question, you will write a recursive descent parser for the following grammar:

$$S \rightarrow E\ a\ E \mid h$$
$$E \rightarrow F\ o \mid g$$
$$F \rightarrow \epsilon \mid f$$

(a) (5 points) Calculate the FIRST sets for each nonterminal in the grammar

```
FIRST(S) = {h,f,o,g}
FIRST(E) = {f,o,g}
FIRST(F) = {epsilon,f}
```

(b) (5 points) For this part, you will implement pseudocode that parses the nonterminals E and F. Your code should use the functions peek(), consume(tok) with a specific literal as the token, and bad_parse(), which triggers a parse error.

```
def parse_S():
  if (peek()=='f' || peek()=='o' || peek()=='g'):
    parse_E()    # parse E
    consume('a') # expect 'a' next
    parse_E()    # parse another E, return
  elif(peek()=='h'):
    consume('h') # Expect 'h'
  else:
    # Call bad_parse() to trigger a parse error
    bad_parse()
```

Write two functions, parse_E() and parse_F(), which parse the nonterminals E and F respectively. Ensure that your functions are compatible with the code below for parse_S().

```
def parse_E():
  if(peek()=='g'):
    consume('g')
  elif(peek()=='f'||peek()=='o'):
    parse_F()
    consume('o')
  else:
    parse_error()

def parse_F():
  if(peek()=='f'):
    consume('f')
  elif(peek()=='o')
    return # no call to consume
  else:
    parse_error() # should only see f/o
```